

AI-Powered Automotive & Car Insurance Market Intelligence Platform

System Architecture Design Document

Version: 1.0 | Date: 2026-03-05 | Author: Senior Software Architect

Table of Contents

- System Overview
- Architectural Layers
- Component Descriptions
- Database Design Overview
- Data Pipeline Flow
- Folder Structure
- Scalability Considerations
- Future Improvements

1. System Overview

Purpose

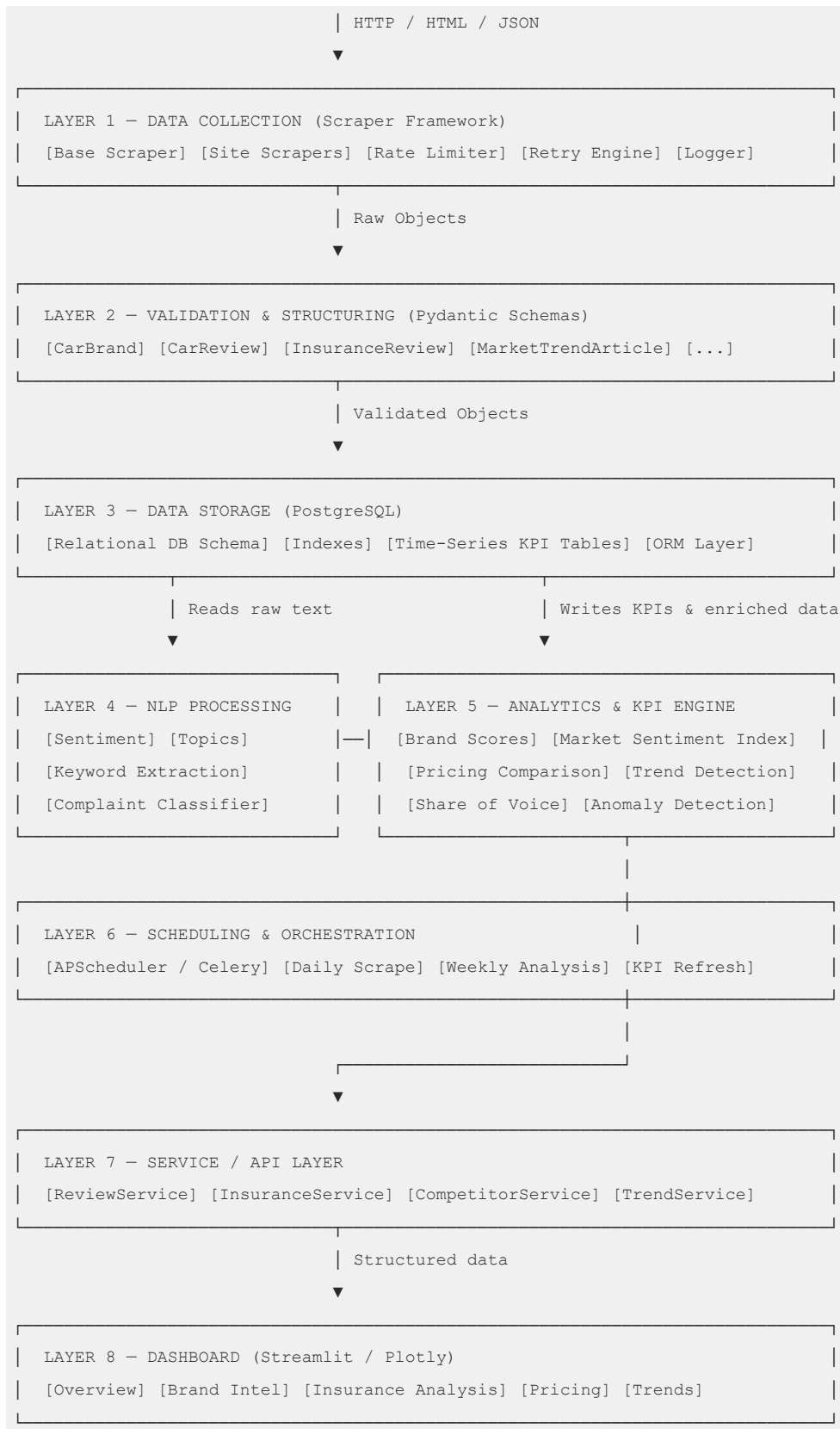
The AI-Powered Automotive & Car Insurance Market Intelligence Platform is a continuously operating data intelligence system that autonomously harvests, processes, and transforms raw web data from automotive and insurance sources into structured business intelligence. The platform surfaces actionable insights via a real-time dashboard for market analysts, product managers, and business strategists.

Core Value Proposition

Capability	Description
Continuous Monitoring	Automated, scheduled scraping of dozens of sources 24/7
AI-Driven Analysis	NLP pipelines for sentiment, topic, and complaint classification
Competitive Intelligence	Cross-brand and cross-insurer pricing and sentiment comparison
Historical Trending	Time-series KPI storage to observe market evolution
Unified Dashboard	Single-pane-of-glass visualization across all intelligence domains

High-Level Architecture Diagram





2. Architectural Layers

Layer 1 — Data Collection

Responsibility: Autonomously harvesting raw data from external online sources.

Design Pattern: Abstract Base Class + Template Method Pattern. A BaseScraper defines the contract (fetch → parse → extract → return). Each site implements its own parsing logic by subclassing.

Key Design Decisions:

- Stateless scrapers: Each scraper run is independent and idempotent, enabling safe retries.
- Configurable rate limiting: Per-domain delay configurations loaded from a central config file.
- Rotating user-agents: A pool of browser-like User-Agent strings is used to reduce detection.
- robots.txt compliance: Scrapers respect crawl delays and disallowed paths before crawling.
- Dual rendering support: Static pages use httpx/requests; JavaScript-heavy pages fall back to Playwright.

Scraper Categories:

Scraper Class	Target Source Type	Rendering
ReviewSiteScraper	Car review aggregators	Static
NewsBlogScraper	Automotive news & blogs	Static
MarketplaceScraper	Car listing marketplaces	JS (Playwright)
InsuranceSiteScraper	Insurer websites, review portals	Static / JS
ForumScraper	Reddit, car forums, community boards	Static
PricingPageScraper	Insurance policy pricing tiers	JS (Playwright)
TrendArticleScraper	Industry trend publications	Static

Cross-Cutting Concerns:

- Retry logic: Exponential backoff with configurable maximum retries (default: 3).
- Timeout management: Per-request and per-session timeouts.
- Structured logging: Each scraper emits JSON logs (source, status, record count, error if any).
- Error isolation: Individual scraper failures do NOT halt the pipeline. Exceptions are caught, logged, and reported.

Layer 2 — Data Validation & Structuring

Responsibility: Ensuring all scraped data is clean, typed, and schema-conformant before storage.

Design Pattern: DTO (Data Transfer Object) + Schema Validation. Pydantic models serve as both the validation layer and the data transfer contract between the scraper and storage layers.

Schema Models:

Model	Key Fields
CarBrand	id, name, country_of_origin, founded_year, logo_url
CarModel	id, brand_id (FK), name, year, segment, body_type
CarReview	id, model_id (FK), source_url, rating, review_text, author, review_date, scraped_at
InsuranceCompany	id, name, country, website, founded_year
InsurancePolicy	id, company_id (FK), policy_name, coverage_type, price_range_min, price_range_max, s
InsuranceReview	id, company_id (FK), source_url, rating, review_text, author, review_date, scraped_at
CompetitorPricing	id, company_id (FK), policy_id (FK), price, currency, snapshot_date
MarketTrendArticle	id, title, source_url, publication_date, body_text, tags, scraped_at

Validation Rules:

- Ratings are normalized to a 0–10 float scale regardless of source.
 - URLs are validated as well-formed HTTP/HTTPS strings.
 - Dates are parsed into timezone-aware datetime objects.
 - Optional fields use strict None defaults rather than empty strings.
 - Duplicate detection uses a composite hash of (source_url + review_date) to prevent re-insertion.
-

Layer 3 — Data Storage

Responsibility: Persistently storing all raw, enriched, and processed data with high query performance.

Primary Database: PostgreSQL 15+

ORM Layer: SQLAlchemy (with Alembic for migrations)

Key Design Principles:

- 3rd Normal Form (3NF): All non-key attributes depend only on the primary key.
- UUID primary keys: Globally unique IDs across all tables for future distributed compatibility.
- Soft deletes: Records are never hard-deleted; a deleted_at timestamp is set instead.
- Audit timestamps: Every table has created_at and updated_at columns managed automatically.
- Partitioning: The car_reviews, insurance_reviews, and kpi_metrics tables are partitioned by month for efficient time-series queries.

Index Strategy:

- B-Tree indexes on all foreign keys.
 - GIN indexes on review_text columns for full-text search.
 - Composite index on (company_id, snapshot_date) for pricing timeline queries.
 - Partial indexes on is_processed = FALSE for efficient NLP queue queries.
-

Layer 4 — NLP Processing

Responsibility: Transforming unstructured text into structured AI-derived signals.

Processing Philosophy: The NLP layer is a stateless processing pipeline — it reads raw text from the database, enriches it, and writes results back. It does not own any data.

NLP Capabilities:

Capability	Method	Output
Sentiment Analysis	Pre-trained transformer (e.g., cardiffnlp/twitter-roberta)	positive / neutral / negative + confidence score
Keyword Extraction	KeyBERT or RAKE	Top-N keywords per review
Topic Modeling	BERTopic or LDA	Topic clusters per corpus
Complaint Classification	Fine-tuned classifier	Categories: pricing, service, reliability, claims, coverage
Trend Detection	Time-windowed frequency analysis	Emerging keyword terms over rolling 7/30-day windows
Anomaly Detection	Z-score on rolling rating averages	Flags sudden drops or spikes in brand/insurer scores

Pipeline Design:

Queue-based intake: Unprocessed records are fetched in configurable batch sizes.

Parallel processing: Multiple NLP workers process batches concurrently (ProcessPoolExecutor or Celery workers).

Result persistence: Enriched fields (sentiment_label, sentiment_score, keywords, topic_id, complaint_category) are written to dedicated enrichment columns.

Model versioning: Each NLP result row stores the model version used, enabling reproducibility and model upgrade tracking.

Layer 5 — Analytics & KPI Engine

Responsibility: Computing, aggregating, and historically storing all business metrics.

Design Pattern: Metric Registry — each KPI is defined as a named computation unit that reads from enriched DB views and writes to the kpi_metrics time-series table.

KPI Catalog:

KPI	Formula	Granularity
avg_rating_by_brand	AVG(rating) grouped by brand	Daily
avg_rating_by_model	AVG(rating) grouped by model	Daily
insurance_satisfaction_score	Weighted avg of star ratings + sentiment scores	Weekly
complaint_frequency_by_category	COUNT(*) grouped by complaint_category	Weekly
pricing_spread_by_insurer	MAX(price) - MIN(price) per policy tier	Daily
market_sentiment_index	Avg sentiment score across all automotive review	Daily
share_of_voice	COUNT(brand_mentions) / total_mentions * 100	Weekly
emerging_trend_score	Keyword frequency delta over rolling windows	Weekly
reliability_index	Weighted avg of complaint-adjusted ratings	Weekly
feature_gap_score	Frequency of feature-related negative keywords	Monthly

Historical Storage Model:

The kpi_metrics table stores every computed KPI as a row with (kpi_name, entity_id, entity_type, value, computed_at). This enables full historical replay and trend charting over any time horizon.

Layer 6 — Automation & Scheduling

Responsibility: Orchestrating all pipeline tasks on defined schedules.

Technology Choices:

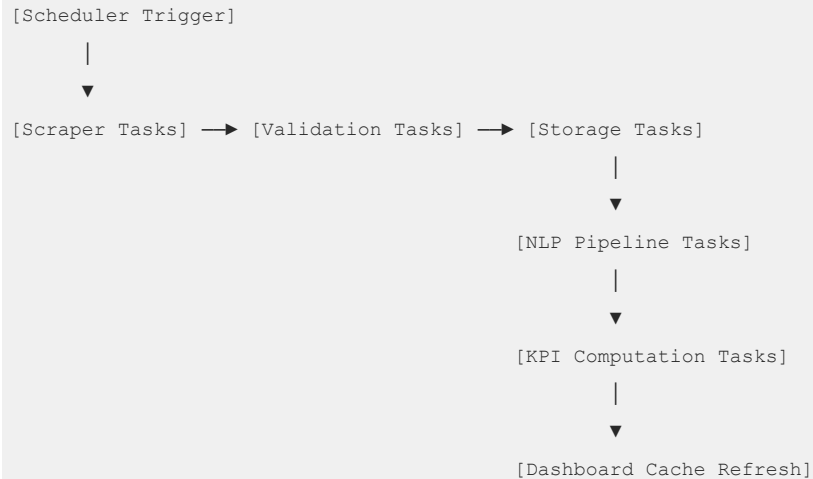
- APScheduler (for single-server deployments): Python-native, low overhead.
- Celery + Redis (for distributed/production deployments): Full task queue with retry, priority, and monitoring support.
- Flower (optional): Web UI for Celery task monitoring.

Schedule Design:

Task	Schedule	Priority
run_all_scrapers	Daily at 02:00 UTC	High
validate_and_store_raw_data	Daily at 03:00 UTC	High
run_nlp_pipeline	Daily at 04:00 UTC	Medium
compute_daily_kpis	Daily at 06:00 UTC	Medium

run_trend_analysis	Weekly (Monday 05:00 UTC)	Low
compute_weekly_kpis	Weekly (Monday 06:00 UTC)	Low
database_cleanup	Weekly (Sunday 01:00 UTC)	Low
export_kpi_snapshots	Monthly (1st, 07:00 UTC)	Low

Pipeline Orchestration Flow:



Error Handling in Scheduling:

- Failed tasks emit alerting logs and can optionally trigger email/Slack notifications.
- Task retries use exponential backoff (3 attempts max).
- A run audit table (pipeline_runs) tracks each job: start time, end time, status, record counts.

Layer 7 — Service / API Layer

Responsibility: Providing a clean, decoupled interface to the data and analytics engine.

Design Pattern: Service Layer (façade over repository + analytics). Services encapsulate all business logic and hide database query complexity from consumers (dashboard, future APIs).

Services:

Service	Responsibilities
ReviewAnalysisService	Fetch reviews by brand/model, aggregated sentiment, rating distributions
InsuranceInsightsService	Insurance ratings, complaint breakdowns, satisfaction trends
CompetitorMonitoringService	Price comparison tables, competitor pricing timelines
TrendAnalysisService	Trending keywords, emerging topics, sentiment shifts over time
KPIService	Retrieve computed KPIs by name, entity, and time range
SearchService	Full-text search across reviews and articles
DataExportService	Export datasets as CSV/JSON for external consumption

Interaction Model:

- Services are read-oriented — they query the database and analytics tables.
- Each service wraps a repository class that owns the raw ORM queries.
- Services apply any filtering, formatting, and business logic before returning results.

- All service methods are designed to be easily wrapped by a REST API (FastAPI) or called directly by Streamlit dashboard pages.

Optional REST API Exposure (FastAPI):

- GET /api/v1/brands/{brand_id}/review-summary
 - GET /api/v1/insurance/{company_id}/sentiment-trend
 - GET /api/v1/competitor/pricing?policy_type=comprehensive
 - GET /api/v1/trends/keywords?window=30d
 - GET /api/v1/kpi/{kpi_name}?entity_id=...&from=...&to=...
-

Layer 8 — Data Visualization Dashboard

Responsibility: Presenting all intelligence in a navigable, interactive, self-updating dashboard.

Technology: Streamlit (primary) with Plotly for interactive charts.

Dashboard Pages:

Page 1: Overview

- Platform health status (last scrape time, record counts, pipeline status)
- Top-level KPI cards: Market Sentiment Index, Top Rated Brand, Most Complained Insurer
- Mini trend charts for each major KPI (7-day sparklines)
- Recent activity feed (latest scraped articles, reviews)

Page 2: Car Brand Intelligence

- Brand ranking table (avg rating, sentiment score, review volume)
- Sentiment distribution donut chart per brand
- Rating trend line chart (30/90/180 day windows)
- Top keywords and complaint categories per brand
- Feature gap heatmap (positive vs. negative features by brand)

Page 3: Car Model Analysis

- Model comparison table with side-by-side metrics
- Model reliability index trend chart
- Review summary word cloud
- Complaint category breakdown bar chart
- Filter by brand, year, segment

Page 4: Insurance Market Analysis

- Insurer ranking (satisfaction score, complaint frequency)
- Satisfaction score trend over time
- Complaint category distribution (coverage, claims, pricing, service)
- Customer sentiment comparison: insurer A vs. B vs. C
- Policy type analysis (comprehensive vs. third-party pricing)

Page 5: Competitor Pricing

- Pricing comparison table (insurer × policy tier × price range)
- Price evolution line chart per insurer (historical snapshots)

- Pricing spread bar chart (cheapest vs. most expensive per coverage type)
- Market share of voice by insurer

Page 6: Market Trends

- Trending keyword cloud (auto-refreshed from trend detection engine)
- Topic modeling visualization (BERTopic cluster map or bubble chart)
- Share of voice evolution stacked area chart
- Emerging trend articles feed (title, source, date, topic tags)
- Anomaly alerts panel (sudden rating drops, pricing spikes)

Dashboard Design Principles:

- Auto-refresh: Configurable refresh interval (default: 60 minutes) pulls latest KPIs.
- Date range filter: Global date range selector affecting all charts.
- Responsive caching: Service results are cached with TTL to avoid redundant DB queries.
- Export buttons: Each chart/table has a CSV/PNG export option.

3. Component Descriptions

Core Components Summary

Component	Layer	Technology	Role
BaseScraper	Collection	Python/httpx/Playwright	Abstract scraping contract
RateLimiter	Collection	asyncio / time.sleep	Per-domain throttling
RetryEngine	Collection	tenacity library	Exponential backoff retry
SchemaValidator	Validation	Pydantic v2	Data contract enforcement
DatabaseClient	Storage	SQLAlchemy + psycopg2	ORM-based DB access
MigrationManager	Storage	Alembic	Schema versioning
SentimentAnalyzer	NLP	Transformers / VADER	Text sentiment scoring
KeywordExtractor	NLP	KeyBERT / RAKE	Term extraction
TopicModeler	NLP	BERTopic	Unsupervised topic clustering
KPIComputer	Analytics	Pandas + SQLAlchemy	Metric computation engine
Scheduler	Automation	APScheduler / Celery	Task orchestration
ServiceLayer	Services	Pure Python	Business logic façade
Dashboard	Visualization	Streamlit + Plotly	Interactive UI
ConfigManager	Cross-cutting	pydantic-settings	Env-var driven config
AuditLogger	Cross-cutting	structlog / loguru	JSON structured logging

4. Database Design Overview

Entity Relationship Overview


```

car_brands (1) ———< (N) car_models
car_models (1) ———< (N) car_reviews
car_reviews (1) ———< (1) review_nlp_results

insurance_companies (1) ———< (N) insurance_policies
insurance_companies (1) ———< (N) insurance_reviews
insurance_reviews (1) ———< (1) review_nlp_results
insurance_policies (1) ———< (N) competitor_pricings

market_trend_articles (1) ———< (1) article_nlp_results

kpi_metrics (N) — stores all computed KPIs by entity
pipeline_runs (N) — audit log of all scheduled runs

```

Core Tables

car_brands

Column	Type	Notes
id	UUID PK	
name	VARCHAR(100) UNIQUE NOT NULL	
country_of_origin	VARCHAR(100)	
founded_year	INTEGER	
logo_url	TEXT	
created_at	TIMESTAMPTZ	Auto-set
updated_at	TIMESTAMPTZ	Auto-updated

car_models

Column	Type	Notes
id	UUID PK	
brand_id	UUID FK → car_brands	CASCADE DELETE
name	VARCHAR(100) NOT NULL	
year	INTEGER	
segment	VARCHAR(50)	e.g., SUV, Sedan
body_type	VARCHAR(50)	

car_reviews

Column	Type	Notes
id	UUID PK	
model_id	UUID FK → car_models	
source_url	TEXT UNIQUE NOT NULL	
source_name	VARCHAR(100)	e.g., "CarAndDriver"
rating	NUMERIC(4,2)	Normalized 0–10

review_text	TEXT	
author	VARCHAR(200)	
review_date	DATE	
scraped_at	TIMESTAMPTZ	
is_processed	BOOLEAN DEFAULT FALSE	NLP queue flag

review_nlp_results

Column	Type	Notes
id	UUID PK	
review_id	UUID FK (polymorphic via type)	
review_type	VARCHAR(20)	'car' or 'insurance'
sentiment_label	VARCHAR(20)	positive/neutral/negative
sentiment_score	NUMERIC(5,4)	0.0 – 1.0
keywords	TEXT[]	PostgreSQL array
topic_id	INTEGER	FK → topics table
complaint_category	VARCHAR(50)	
model_version	VARCHAR(50)	NLP model version tag
processed_at	TIMESTAMPTZ	

kpi_metrics

Column	Type	Notes
id	UUID PK	
kpi_name	VARCHAR(100) NOT NULL	e.g., "avg_rating_by_brand"
entity_type	VARCHAR(50)	'brand', 'model', 'insurer'
entity_id	UUID	
value	NUMERIC(12,4)	Computed metric value
computed_at	TIMESTAMPTZ	Partition key
granularity	VARCHAR(20)	'daily', 'weekly', 'monthly'

pipeline_runs

Column	Type	Notes
id	UUID PK	
task_name	VARCHAR(100)	e.g., "run_all_scrapers"
started_at	TIMESTAMPTZ	
finished_at	TIMESTAMPTZ	
status	VARCHAR(20)	success / failed / partial
records_scraped	INTEGER	
records_stored	INTEGER	
error_message	TEXT	NULL if success

Normalization Strategy

- 1NF: All columns are atomic; arrays use proper PostgreSQL array types or JSON.
- 2NF: No partial dependencies; all non-key attributes are fully dependent on the whole key.
- 3NF: No transitive dependencies; brand country is in car_brands, not car_reviews.
- Denormalization: The kpi_metrics table is intentionally denormalized (flat key-value pairs) to optimize time-series reads.

5. Data Pipeline Flow

End-to-End Data Journey

```

[External Source]
  |
  | HTTP GET (with rate limiting, retries, user-agent rotation)
  ▼
[Scraper Module]
  |
  | Raw HTML / JSON dict
  ▼
[Parser Module] - extracts structured fields from raw content
  |
  | Raw Python dict
  ▼
[Schema Validator (Pydantic)]
  |— INVALID → [Dead Letter Log] → stored for manual review
  |
  | Validated domain object
  ▼
[Duplicate Checker] - hash-based dedup against DB
  |— DUPLICATE → Skip (log warning)
  |
  | New record
  ▼
[Database Writer] - stores to appropriate table via ORM
  |
  | (Async, nightly) - NLP queue reads unprocessed records
  ▼
[NLP Pipeline]
  | Batch read from DB where is_processed = FALSE
  |
  |→ Sentiment Analyzer → sentiment_label, sentiment_score
  |→ Keyword Extractor  → keywords[]
  |→ Topic Modeler      → topic_id
  |→ Complaint Classifier → complaint_category
  |
  | Write enrichment to review_nlp_results

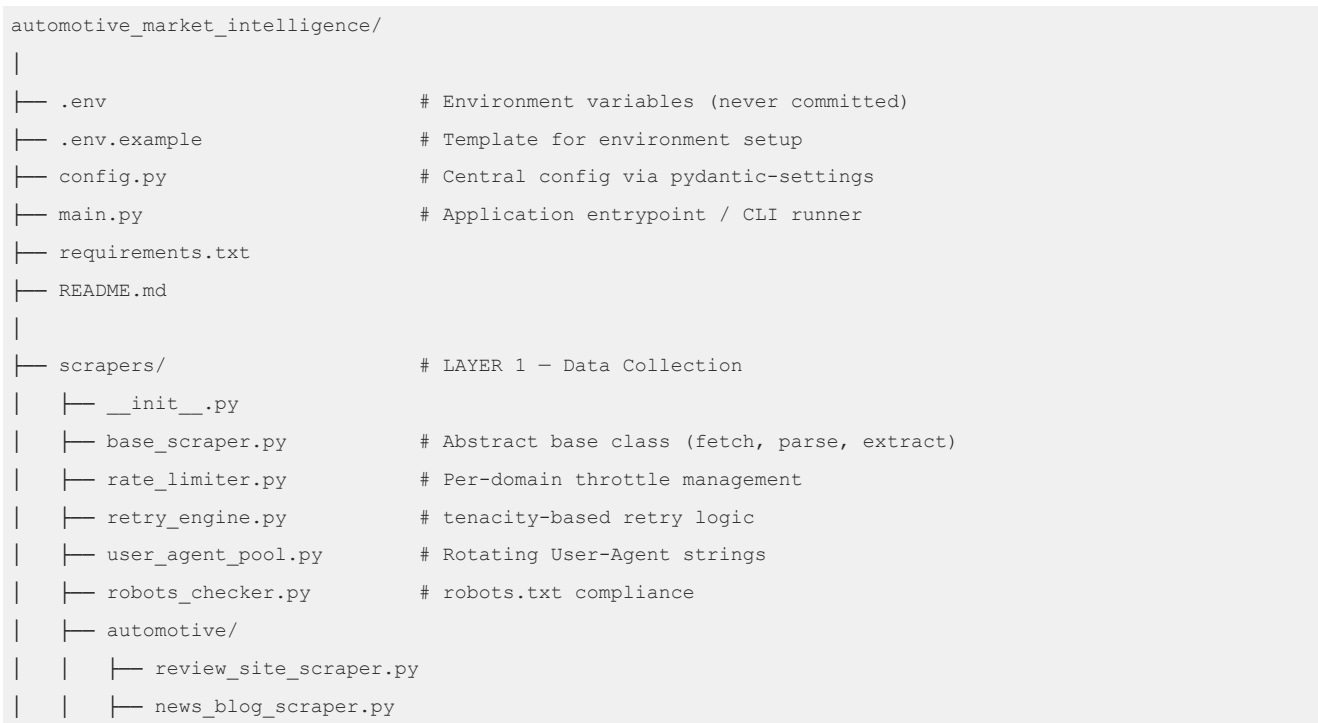
```



Data Freshness Guarantees

Data Type	Freshness Target
Raw reviews	< 24 hours
Sentiment signals	< 24 hours post-scrape
Daily KPIs	< 6 hours after scrape completes
Weekly KPIs / Trends	< 24 hours after weekly job
Dashboard display	Refreshed every 60 minutes

6. Folder Structure



```

| | └─ marketplace_scraper.py
| | └─ trend_article_scraper.py
| └─ insurance/
|     └─ insurance_site_scraper.py
|     └─ pricing_page_scraper.py
|     └─ forum_scraper.py
|
└─ schemas/                                # LAYER 2 - Validation & Structuring
    └─ __init__.py
    └─ automotive.py                      # CarBrand, CarModel, CarReview
    └─ insurance.py                       # InsuranceCompany, InsurancePolicy, InsuranceReview
    └─ market.py                         # MarketTrendArticle, CompetitorPricing
    └─ nlp_results.py                    # ReviewNLPResult, ArticleNLPResult
|
└─ database/                              # LAYER 3 - Data Storage
    └─ __init__.py
    └─ models.py                         # SQLAlchemy ORM model definitions
    └─ connection.py                    # DB engine, session factory
    └─ repositories/
        └─ car_repository.py            # CRUD for car brands/models/reviews
        └─ insurance_repository.py
        └─ market_repository.py
        └─ kpi_repository.py
    └─ migrations/                      # Alembic migration scripts
        └─ env.py
        └─ alembic.ini
        └─ versions/
|
└─ nlp/                                    # LAYER 4 - NLP Processing
    └─ __init__.py
    └─ pipeline.py                      # Orchestrates all NLP steps per batch
    └─ sentiment_analyzer.py            # Transformer-based sentiment scoring
    └─ keyword_extractor.py             # KeyBERT / RAKE extractor
    └─ topic_modeler.py                 # BERTopic model wrapper
    └─ complaint_classifier.py          # Fine-tuned complaint category classifier
    └─ trend_detector.py                # Rolling window keyword frequency analysis
    └─ anomaly_detector.py              # Z-score anomaly flagging on KPI streams
    └─ models/                          # Stored model artifacts / weights
|
└─ analytics/                             # LAYER 5 - KPI Engine
    └─ __init__.py
    └─ kpi_engine.py                    # KPI Registry and computation dispatcher
    └─ automotive_kpis.py                # Brand/model KPI implementations
    └─ insurance_kpis.py                 # Insurance satisfaction, complaint KPIs
    └─ market_kpis.py                   # Share of voice, pricing spread KPIs
    └─ trend_kpis.py                     # Emerging trend scoring
|
└─ scheduler/                             # LAYER 6 - Automation
    └─ __init__.py
    └─ scheduler.py                     # APScheduler or Celery setup

```

```

|   └─ tasks/
|   |   └─ scraping_tasks.py      # run_all_scrapers task
|   |   └─ nlp_tasks.py          # run_nlp_pipeline task
|   |   └─ kpi_tasks.py          # compute_*_kpis tasks
|   |   └─ maintenance_tasks.py  # database_cleanup task
|   └─ celery_app.py             # Celery application factory (optional)
|
└─ services/                     # LAYER 7 - Service Layer
    └─ __init__.py
    └─ review_analysis_service.py
    └─ insurance_insights_service.py
    └─ competitor_monitoring_service.py
    └─ trend_analysis_service.py
    └─ kpi_service.py
    └─ search_service.py
    └─ data_export_service.py
|
└─ api/                          # Optional REST API (FastAPI)
    └─ __init__.py
    └─ main.py                   # FastAPI app factory
    └─ routers/
        └─ brands.py
        └─ insurance.py
        └─ trends.py
        └─ kpis.py
    └─ dependencies.py          # Auth, DB session injection
|
└─ dashboard/                   # LAYER 8 - Visualization
    └─ app.py                   # Streamlit entrypoint
    └─ pages/
        └─ 1_Overview.py
        └─ 2_Car_Brand_Intelligence.py
        └─ 3_Car_Model_Analysis.py
        └─ 4_Insurance_Market_Analysis.py
        └─ 5_Competitor_Pricing.py
        └─ 6_Market_Trends.py
    └─ components/
        └─ charts.py            # Reusable Plotly chart builders
        └─ tables.py            # Reusable styled table builders
        └─ kpi_cards.py         # KPI metric card components
        └─ filters.py           # Global date range, brand filters
    └─ cache.py                 # Streamlit caching decorators
|
└─ tests/                      # Test Suite
    └─ unit/
        └─ test_scrapers.py
        └─ test_schemas.py
        └─ test_nlp.py
        └─ test_kpi_engine.py
    └─ integration/

```

```
| | └─ test_pipeline.py
| | └─ test_database.py
| └─ fixtures/
|     └─ sample_data.json      # Mock scrape outputs for testing
|
└─ logs/                      # Runtime logs (gitignored)
    └─ scraper.log
    └─ nlp.log
    └─ scheduler.log
    └─ error.log
```

Folder Role Summary

Folder	Role
scrapers/	All web data acquisition logic, organized by domain
schemas/	Pydantic data contracts for validation and type safety
database/	ORM models, connection management, repositories, migrations
nlp/	All AI/NLP processing modules and model artifacts
analytics/	KPI computation registry and business metric implementations
scheduler/	Task scheduling, pipeline orchestration, cron-like task definitions
services/	Business logic layer consumed by dashboard and optional API
api/	Optional FastAPI REST interface for external consumers
dashboard/	Streamlit multi-page dashboard with reusable chart components
tests/	Unit, integration, and fixture-based test coverage
logs/	Runtime logs emitted by all layers (gitignored)

7. Scalability Considerations

Horizontal Scaling

Concern	Strategy
Scraping throughput	Add more scraper worker processes/containers; each scraper is stateless
NLP processing	Celery workers scale horizontally; GPU workers for transformer inference
Database reads	Add read replicas for dashboard and API queries
Database writes	Use connection pooling (PgBouncer) to handle concurrent write bursts
Scheduling	Move from APScheduler to Celery Beat for distributed task coordination

Reliability Practices

- Dead letter queues: Failed scrape records are stored for manual inspection, not silently dropped.
- Pipeline idempotency: Re-running any pipeline task doesn't create duplicate data (dedup enforced at storage layer).
- Graceful degradation: If NLP service is unavailable, raw data still gets stored; NLP runs when restored.

- Health checks: Each service module exposes a health-check method for monitoring integrations.

Performance Optimizations

- Batch NLP processing: Reviews are processed in configurable batches (default 500) rather than one-by-one.
- Materialized views: Frequently used aggregations (brand rankings, sentiment averages) are stored as PostgreSQL materialized views and refreshed on schedule.
- Streamlit caching: `@st.cache_data` with TTL prevents redundant service calls during dashboard interactions.
- Async scraping: Scrapers use `asyncio` with `httpx.AsyncClient` to pipeline multiple HTTP requests concurrently.
- Dashboard lazy loading: Heavy charts are loaded asynchronously on tab selection, not all at page load.

Configuration Management

- All environment-specific settings are managed through `.env` files.
- `pydantic-settings` validates and types the configuration at startup.
- Separate configuration profiles for development, staging, production environments.
- Secrets (DB passwords, API keys) are NEVER hardcoded; always read from environment.

Observability Stack

Signal	Tool
Structured Logging	loguru or structlog — JSON logs per component
Pipeline Audit	pipeline_runs DB table tracks every job execution
Error Alerting	Log-level ERROR triggers Slack/email notification (optional)
Metrics	prometheus_client (optional) for operational dashboards
Tracing	OpenTelemetry (future) for distributed trace correlation

8. Future Improvements

Short-Term (3–6 months)

Improvement	Impact
Authentication & multi-user dashboard	Allow role-based access (analyst, admin, viewer)
Real-time scraping via webhooks	React to news articles within minutes instead of daily
Additional source connectors	YouTube comment analysis, Twitter/X automotive discussions
A/B sentiment model comparison	Benchmark multiple NLP models and let operators choose
Data quality scoring	Each scraped record gets a quality confidence score

Medium-Term (6–12 months)

Improvement	Impact
LLM-powered review summarization	Auto-generate executive summaries per brand/insurer per week
Predictive pricing model	ML model to forecast insurance price trends from historical data

GraphQL API	More flexible querying for external consumers
Multi-language support	Scrape and analyze non-English reviews (French, Arabic, etc.)
Competitor alerting	Automatic alert when a competitor drops/raises prices significantly

Long-Term (12+ months)

Improvement	Impact
Distributed scraping cluster	Kubernetes-orchestrated scraper fleet for massive scale
Vector database integration	Semantic search over all reviews using embeddings
Custom fine-tuned NLP models	Domain-specific automotive/insurance sentiment models
Interactive "ask the data" interface	Natural language Q&A over the intelligence platform using RAG
White-label SaaS mode	Multi-tenant deployment for different market verticals

Document Status: Enhanced with 6 additional architectural components — v1.1

Next Step: Translate this architecture into an implementation plan with sprint-level task breakdown.

Architecture Enhancements

Version: 1.1 | Date: 2026-03-06 | Status: Addendum — Existing Sections Unchanged

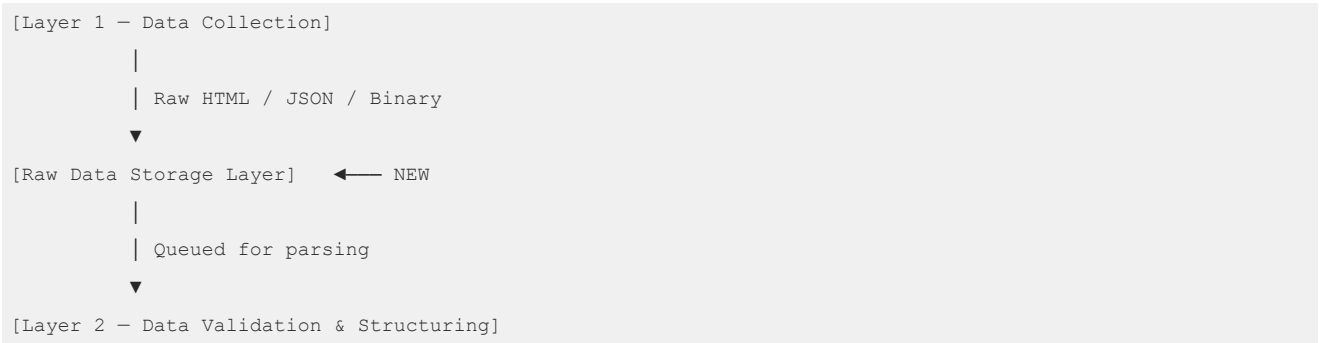
The following sections extend the original architecture without modifying any prior content.

Each enhancement adds a missing engineering component to the platform design.

Enhancement 1 — Raw Data Storage Layer

Position in Architecture

This layer is inserted between Layer 1 (Data Collection) and Layer 2 (Data Validation & Structuring). It acts as a persistent landing zone for all scraped content before any parsing or validation occurs.



Purpose & Motivation

The Raw Data Storage Layer addresses a critical gap in the original architecture: the loss of original scraped content after parsing. Without this layer, if a parser fails, produces incorrect output, or if a future NLP model requires re-processing, the original data is unrecoverable and the scraper must re-run.

Key Benefits:

Benefit	Description
Data Reprocessing	Re-parse historical content when schemas or parsers are updated, without re-scraping
Debugging	Inspect exact raw HTML/JSON that caused a parse failure or validation error
Improved NLP	Run new or improved NLP models over historical raw content on demand
Data Auditing	Prove what was scraped, when, and from which source at any point in time
Source Archival	Preserve a snapshot of the internet as seen at scraping time

Database Tables

raw_pages

Stores the full HTML content of every scraped page.

Column	Type	Notes
id	UUID PK	
scrape_job_id	UUID FK → scraping_jobs	Links to the job that produced this page
source_url	TEXT NOT NULL	The exact URL scraped
source_domain	VARCHAR(200)	Extracted domain for partitioning/indexing
http_status_code	INTEGER	e.g., 200, 403, 429
raw_html	TEXT	Full page HTML content
content_hash	VARCHAR(64)	SHA-256 of content to detect changes
scraped_at	TIMESTAMPTZ	When the page was fetched
is_parsed	BOOLEAN DEFAULT FALSE	Parsing queue flag
parse_error	TEXT	NULL if successfully parsed
scraper_version	VARCHAR(50)	Version of the scraper that produced this

raw_api_responses

Stores raw JSON responses from any structured API or JSON endpoint.

Column	Type	Notes
id	UUID PK	
scrape_job_id	UUID FK → scraping_jobs	
endpoint_url	TEXT NOT NULL	API endpoint called
request_params	JSONB	Query parameters sent
response_headers	JSONB	HTTP response headers
response_body	JSONB	Full raw JSON response body
http_status_code	INTEGER	
response_size_bytes	INTEGER	
scraped_at	TIMESTAMPTZ	

is_parsed	BOOLEAN DEFAULT FALSE	
-----------	-----------------------	--

raw_scrape_logs

Stores detailed operational logs for every scraping attempt.

Column	Type	Notes
id	UUID PK	
scrape_job_id	UUID FK → scraping_jobs	
scraper_name	VARCHAR(100)	Class name of the scraper used
target_url	TEXT	URL that was attempted
attempt_number	INTEGER	Which retry attempt (1, 2, 3...)
status	VARCHAR(20)	success, failed, skipped, rate_limited
duration_ms	INTEGER	Time taken for the request
bytes_downloaded	INTEGER	
error_type	VARCHAR(100)	Exception class name if failed
error_message	TEXT	Full error message or traceback
user_agent	TEXT	User-agent string used
logged_at	TIMESTAMPTZ	

Storage Strategy

- Raw HTML and JSON bodies are stored as PostgreSQL TEXT / JSONB columns for simple deployments.
- For large-scale deployments: Binary blobs should be offloaded to object storage (MinIO or AWS S3), with only the object key stored in the database column.
- Retention policy: Raw pages are retained for 90 days by default. A scheduled maintenance_tasks.py job deletes records older than the configured retention window.
- Change detection: The content_hash column on raw_pages allows the system to detect when a page's content has genuinely changed between scraping runs, avoiding redundant re-parsing.

Enhancement 2 — Data Versioning & Historical Tracking

Strategy Overview

Historical tracking is not a single feature — it is a cross-cutting design discipline applied to every data entity in the system. Every record that can change over time must carry version metadata to support time-series analytics and market evolution dashboards.

Versioning Metadata Standard

Every time-sensitive table must include the following standard metadata columns:

Column	Type	Purpose
scraped_at	TIMESTAMPTZ	Exact timestamp when this version of the record was scraped

source	VARCHAR(200)	Source domain or platform name
version	INTEGER	Auto-incremented version number per entity
confidence_score	NUMERIC(4,3)	Data quality confidence: 0.000–1.000
is_latest	BOOLEAN	TRUE only on the most recent version
superseded_at	TIMESTAMPTZ	When this version was superseded by a newer one

Per-Entity Versioning Design

Car Prices (via car_price_history)

A dedicated history table tracks every unique price observation for a car model.

Column	Type	Notes
id	UUID PK	
model_id	UUID FK → car_models	
listed_price	NUMERIC(12,2)	
currency	CHAR(3)	ISO 4217 currency code
price_type	VARCHAR(50)	MSRP, dealer_listed, auction
source	VARCHAR(200)	
scraped_at	TIMESTAMPTZ	
confidence_score	NUMERIC(4,3)	
version	INTEGER	

Insurance Quotes (via insurance_quote_history)

Insurance pricing data is inherently volatile — quotes change frequently by region, age, and vehicle type.

Column	Type	Notes
id	UUID PK	
policy_id	UUID FK → insurance_policies	
quoted_price	NUMERIC(10,2)	
coverage_level	VARCHAR(50)	comprehensive, third-party, etc.
region	VARCHAR(100)	
valid_from	DATE	
valid_to	DATE	NULL if still current
scraped_at	TIMESTAMPTZ	
source	VARCHAR(200)	
version	INTEGER	
confidence_score	NUMERIC(4,3)	

Reviews

Reviews themselves are immutable once published, but their NLP-derived metadata evolves as models are updated. The review_nlp_results table already uses a model_version column to support this pattern — each time a new NLP model is applied, a new row is inserted rather than overwriting the previous result.

Competitor Pricing (via `competitor_pricings`)

Already designed as a snapshot table. Each row is a point-in-time observation. The combination of (`company_id`, `policy_id`, `snapshot_date`) forms a natural composite key for timeline queries.

KPI Metrics (via `kpi_metrics`)

Already designed as an append-only time-series table. Every KPI computation creates a new row — historical values are never overwritten.

Time-Series Analytics Support

The versioning design enables the following analytical patterns:

Pattern	How It Is Supported
Price trend chart	<code>SELECT price, scraped_at FROM car_price_history WHERE model_id = X ORDER BY scraped_at</code>
Insurance quote evolution	Filter <code>insurance_quote_history</code> by policy and date range
KPI trend over 90 days	Query <code>kpi_metrics</code> with <code>computed_at BETWEEN now() - interval '90 days' AND now()</code>
Market sentiment history	Join <code>kpi_metrics</code> WHERE <code>kpi_name = 'market_sentiment_index'</code> ordered by <code>computed_at</code>
Data freshness alert	Compare <code>MAX(scraped_at)</code> per source against expected scraping interval

Confidence Scoring

The `confidence_score` field (0.0–1.0) reflects data quality and is computed from:

- Source reliability score: Known-good sources (official brand sites) score higher than forums.
- Completeness score: Proportion of non-null core fields in the record.
- HTTP status: A 200 response scores higher than a cached/CDN response.
- Validation pass rate: Records requiring coercion during Pydantic validation receive a lower score.

Only records with `confidence_score >= 0.7` are used in KPI computations by default. This threshold is configurable via the central config file.

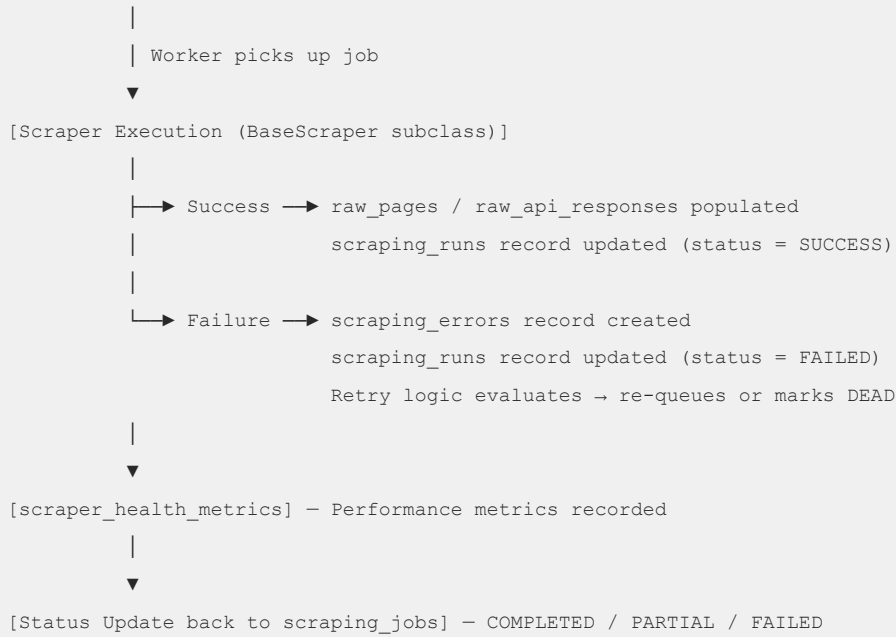
Enhancement 3 — Scraper Job Management System

Purpose

The Scraper Job Management System is a control plane layered on top of the scheduling system. While the scheduler determines `_when_` scrapers run, this subsystem tracks `_what_` each run did, monitors `_health_`, enables `_retries_`, and provides `_observability_` into every scraping operation.

Architecture Flow

```
[Scheduler (APScheduler / Celery Beat)]
    |
    | Triggers job
    ▼
[scraping_jobs] — Job record created with status = QUEUED
```



Database Tables

scraping_jobs

Represents a high-level scraping directive (e.g., "scrape all car review sites").

Column	Type	Notes
id	UUID PK	
job_name	VARCHAR(100)	e.g., daily_automotive_reviews
scraper_class	VARCHAR(200)	Python class path of the scraper
target_url_pattern	TEXT	URL or URL pattern to scrape
scheduled_at	TIMESTAMPTZ	When the scheduler triggered this
priority	INTEGER	1=High, 2=Medium, 3=Low
status	VARCHAR(20)	QUEUED, RUNNING, COMPLETED, FAILED, DEAD
max_retries	INTEGER DEFAULT 3	
retry_count	INTEGER DEFAULT 0	
config_overrides	JSONB	Per-job rate limit, timeout overrides
created_at	TIMESTAMPTZ	
updated_at	TIMESTAMPTZ	

scraping_runs

Represents a single execution attempt of a scraping job.

Column	Type	Notes
id	UUID PK	
job_id	UUID FK → scraping_jobs	
started_at	TIMESTAMPTZ	
finished_at	TIMESTAMPTZ	

status	VARCHAR(20)	SUCCESS, FAILED, PARTIAL, TIMEOUT
pages_fetched	INTEGER	Count of HTTP requests made
records_extracted	INTEGER	Records successfully parsed
records_rejected	INTEGER	Records failing validation
bytes_downloaded	BIGINT	Total data transferred
duration_seconds	NUMERIC(8,2)	
exit_message	TEXT	Summary or error message

scraping_errors

Captures every exception or failure event during scraping for debugging.

Column	Type	Notes
id	UUID PK	
run_id	UUID FK → scraping_runs	
job_id	UUID FK → scraping_jobs	
error_type	VARCHAR(100)	Python exception class name
error_message	TEXT	Full error message
stack_trace	TEXT	Full Python traceback
target_url	TEXT	URL being scraped when error occurred
is_retryable	BOOLEAN	Whether retry logic should re-queue
occurred_at	TIMESTAMPTZ	

scraper_health_metrics

Aggregated performance metrics per scraper, computed after each run.

Column	Type	Notes
id	UUID PK	
scraper_name	VARCHAR(100)	
run_id	UUID FK → scraping_runs	
avg_response_time_ms	NUMERIC(8,2)	Average HTTP response time
success_rate	NUMERIC(5,4)	Successful pages / total attempts
error_rate	NUMERIC(5,4)	Failed pages / total attempts
rate_limit_hits	INTEGER	Count of 429 responses received
robots_blocked_count	INTEGER	URLs skipped due to robots.txt
measured_at	TIMESTAMPTZ	

Capabilities Enabled

Capability	How It Is Implemented
Monitoring	Query scraper_health_metrics for real-time success rates and response times
Retry Management	scraping_jobs.retry_count vs max_retries; failed jobs re-queued with delay
Debugging Failed Scrapers	scraping_errors stores full stack traces linked to the exact run

Performance Tracking	scraper_health_metrics enables trend analysis of scraper degradation over time
Dead Letter Handling	Jobs exceeding max_retries are set to DEAD status and flagged for manual review
SLA Monitoring	Compare scheduled_at vs started_at to detect scheduler latency

Enhancement 4 — End-to-End Data Pipeline Flow

Complete 10-Step Pipeline

The following describes every step of the data journey from external sources to the dashboard, with explicit layer interactions.

Step 1 — Scheduler Triggers Scraping Jobs

- The Scheduler (APScheduler or Celery Beat) fires at the configured time (e.g., 02:00 UTC).
- It creates a new row in scraping_jobs with status QUEUED for each configured scraper.
- Jobs are dispatched to available worker processes or Celery task queue.

Layers involved: Scheduling Layer → Scraper Job Management System

Step 2 — Scrapers Collect Raw Content

- Each scraper subclass is instantiated and begins fetching its target URLs.
- Static pages are fetched via httpx with retry logic from RetryEngine.
- Dynamic pages trigger a Playwright browser session managed by PlaywrightBrowserPool.
- Rate limiting is enforced per domain by RateLimiter.
- robots.txt compliance is checked by RobotsChecker before each domain is accessed.

Layers involved: Data Collection Layer

Step 3 — Raw Content Stored in Raw Data Storage

- Every fetched page or API response is immediately written to raw_pages or raw_api_responses.
- A scraping log entry is written to raw_scrape_logs regardless of success or failure.
- is_parsed is set to FALSE — the parsing queue picks this up asynchronously.
- scraping_runs and scraping_jobs are updated with progress metrics.

Layers involved: Raw Data Storage Layer, Scraper Job Management System

Step 4 — Parsers Extract Structured Data

- A background worker reads raw_pages WHERE is_parsed = FALSE.
- Each scraper's parse() method is called on the stored raw HTML.
- Structured Python dicts are extracted (title, rating, review_text, author, date, URL, etc.).
- If parsing fails, raw_pages.parse_error is populated and is_parsed remains FALSE.

Layers involved: Data Collection Layer (Parser Module), Raw Data Storage Layer

Step 5 — Pydantic Schemas Validate the Data

- The parsed dict is passed through the appropriate Pydantic schema (e.g., CarReview, InsuranceReview).
- Type coercion, normalization (rating scale), and field validation are applied.
- Invalid records are sent to the Dead Letter Log with the validation error message.
- A confidence_score is computed based on validation pass completeness.

Layers involved: Data Validation & Structuring Layer

Step 6 — Structured Data Saved to PostgreSQL

- Validated domain objects are passed to the appropriate Repository class.
- A hash-based duplicate check prevents re-inserting previously seen records.
- New records are inserted via SQLAlchemy ORM with is_processed = FALSE.
- Version metadata (scraped_at, source, version, confidence_score) is populated.
- raw_pages.is_parsed is set to TRUE.

Layers involved: Data Storage Layer (via Repository classes)

Step 7 — NLP Pipeline Processes Text Data

- The NLP pipeline task is triggered by the scheduler (04:00 UTC, after storage is complete).
- It reads all records WHERE is_processed = FALSE in configurable batches.
- Each record's text is passed through the full NLP chain:
- Sentiment Analyzer → sentiment_label, sentiment_score
- Keyword Extractor → keywords[]
- Topic Modeler → topic_id
- Complaint Classifier → complaint_category
- Results are written to review_nlp_results with the model_version tag.
- is_processed is set to TRUE on the source record.

Layers involved: NLP Processing Layer

Step 8 — Analytics Engine Computes KPIs

- Triggered after NLP completes (06:00 UTC).
- The KPIEngine iterates through every registered KPI definition.
- Each KPI queries enriched data via materialized SQLAlchemy views.
- Results are written as new rows to kpi_metrics with computed_at, granularity, and entity_id.
- Historical values are never overwritten — every computation run is preserved.

Layers involved: Analytics & KPI Engine

Step 9 — Services Expose Processed Insights

- Service classes (ReviewAnalysisService, KPIService, etc.) read from the database and KPI tables.
- They format and aggregate results into Python objects or dictionaries.
- Services are consumed directly by Streamlit dashboard pages OR exposed via FastAPI REST endpoints.
- All service results are cached with configurable TTL to reduce repeated database queries.

Layers involved: Service / API Layer

Step 10 — Dashboard Visualizes Results

- Streamlit renders the multi-page dashboard by calling service methods on page load.
- Charts (Plotly) and tables are rendered from the structured data returned by services.
- Global date range and brand/insurer filters propagate across all pages.
- The dashboard auto-refreshes every 60 minutes, re-calling services to surface updated KPIs.
- Export buttons allow users to download chart data as CSV or PNG.

Layers involved: Data Visualization Dashboard

Layer Interaction Summary

Step	Source Layer	Target Layer
1	Scheduling	Job Management
2	Job Management	Data Collection
3	Data Collection	Raw Data Storage
4	Raw Data Storage	Data Collection (Parser)
5	Data Collection	Data Validation
6	Data Validation	Data Storage (PostgreSQL)
7	Data Storage	NLP Processing
8	NLP Processing	Analytics & KPI Engine
9	Analytics Engine	Service Layer
10	Service Layer	Dashboard

Enhancement 5 — Technology Stack Specification

Full Technology Stack

Backend — Core Application

Technology	Version	Role
Python	3.11+	Primary runtime language
FastAPI	0.110+	Optional REST API layer (async, OpenAPI auto-docs)
Pydantic v2	2.x	Schema validation and settings management
pydantic-settings	2.x	Environment variable configuration management
loguru / structlog	Latest	Structured JSON logging

Web Scraping

Technology	Role
Scrapy	High-throughput crawling framework for large-scale link traversal
BeautifulSoup4	HTML parsing for lightweight, targeted scrapers
httpx	Async HTTP client for concurrent requests with timeout control

Playwright	Headless Chromium for JavaScript-rendered pages (SPAs, dynamic pricing)
tenacity	Retry logic with exponential backoff for failed HTTP requests
fake-useragent	Rotating user-agent pool to reduce bot detection

Database & ORM

Technology	Role
PostgreSQL 15+	Primary relational database (all structured, enriched, and KPI data)
SQLAlchemy 2.x	ORM layer (declarative models, async session support)
Alembic	Database migration management and schema versioning
psycopg3	Modern async PostgreSQL driver for SQLAlchemy
PgBouncer	Connection pooling for high-concurrency write workloads

NLP & AI Processing

Technology	Role
spaCy	Named entity recognition, tokenization, linguistic features
HuggingFace Transformers	Pre-trained transformer models (sentiment, classification)
KeyBERT	BERT-powered keyword extraction
BERTopic	Neural topic modeling with embedding-based clustering
RAKE-NLTK	Lightweight keyword extraction fallback
NLTK	Text preprocessing (stopwords, tokenization)

Task Scheduling & Orchestration

Technology	Role
APScheduler	Single-server cron-like scheduling (development/small-scale)
Celery	Distributed task queue for production-scale worker execution
Redis	Celery message broker and result backend
Flower	Web UI for monitoring Celery task queues and worker health

Dashboard & Visualization

Technology	Role
Streamlit	Multi-page dashboard framework (Python-native, no frontend code)
Plotly	Interactive charts (line, bar, donut, heatmap, scatter)
Pandas	Data manipulation for chart preparation and KPI aggregation

Infrastructure & DevOps

Technology	Role
Docker	Containerization of all services (scraper, API, dashboard, scheduler)
Docker Compose	Multi-service local orchestration (PostgreSQL, Redis, app containers)
python-dotenv	.env file loading for local development

pytest	Test framework for unit and integration tests
pytest-asyncio	Async test support for testing async scraper and service code

Logging & Observability

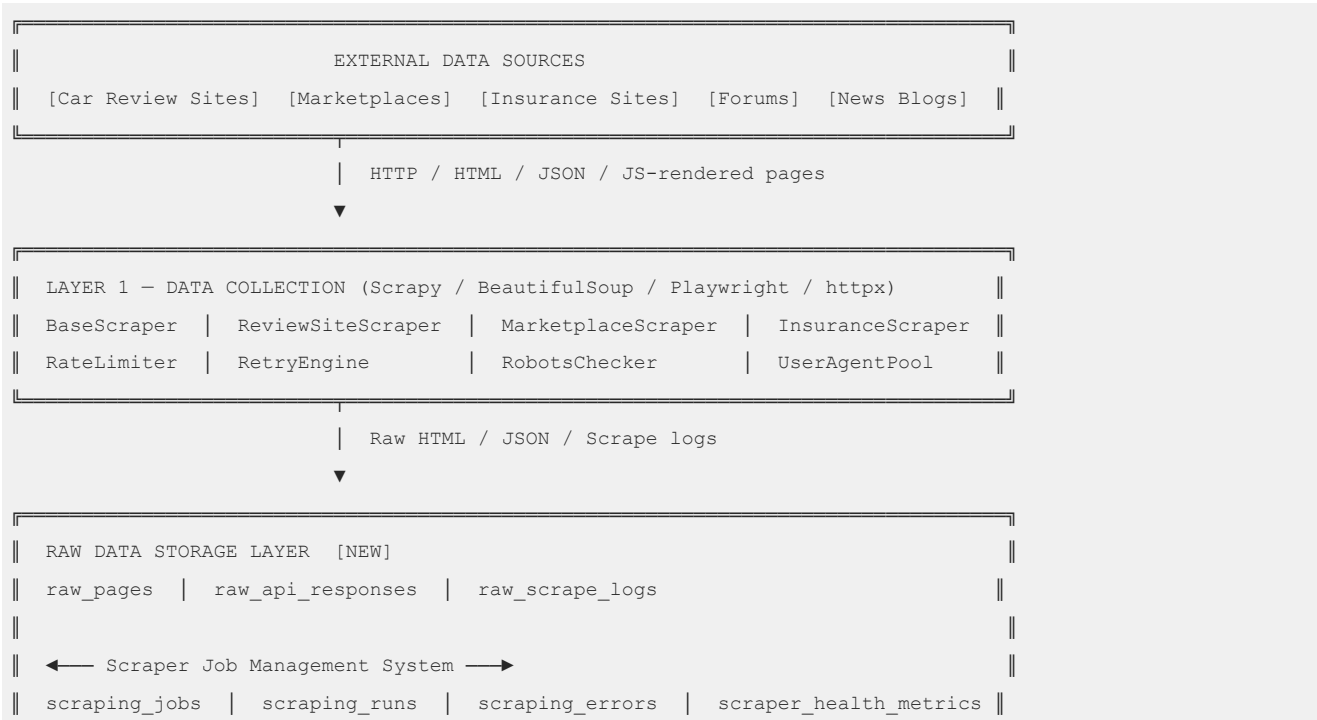
Technology	Role
loguru	Structured file and console logging with automatic rotation
structlog	JSON-formatted log output for machine-readable log ingestion
Prometheus client	Optional metrics endpoint for operational monitoring
OpenTelemetry	Distributed tracing (future enhancement)

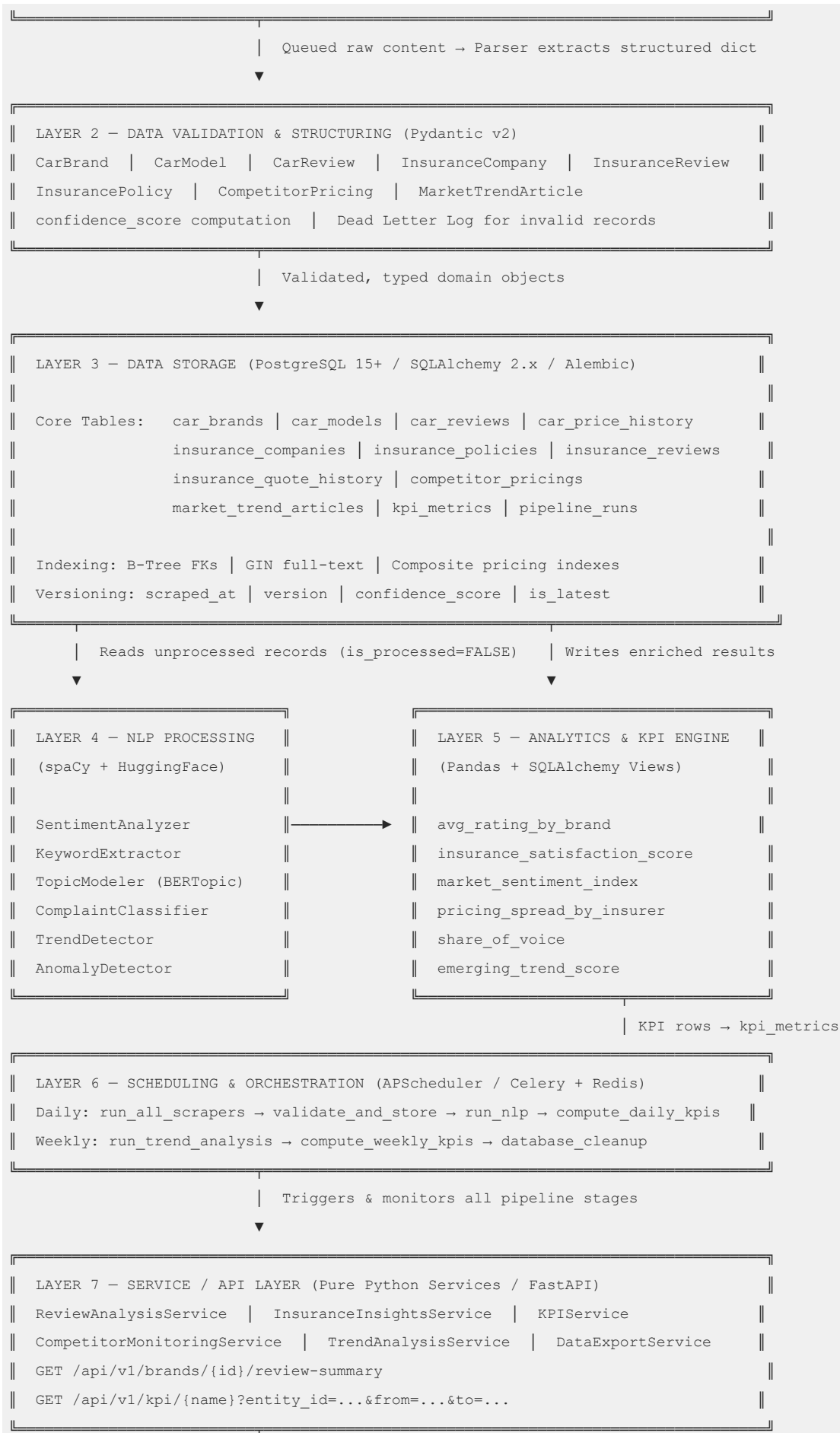
Technology Selection Rationale

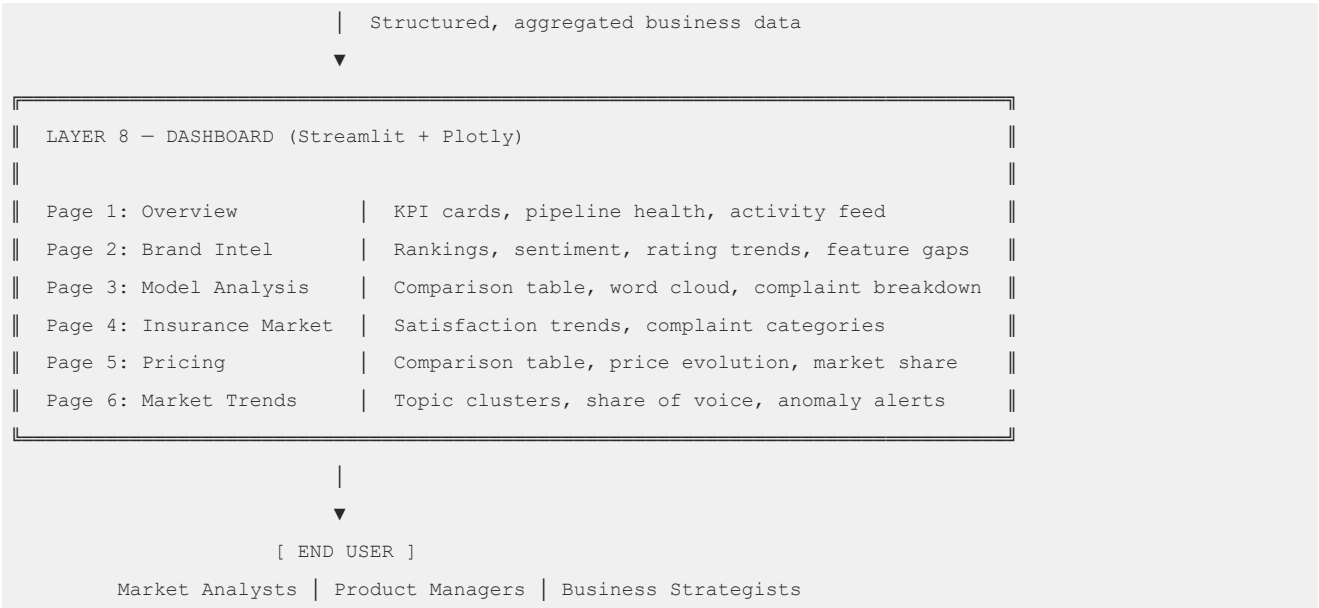
Decision	Rationale
PostgreSQL over MongoDB	Relational data with complex joins suits structured automotive/insurance data better than NoSQL
Streamlit over React/Vue	Python-native dashboard minimizes frontend expertise requirement; faster iteration for analytics
Playwright over Selenium	Better async support, faster execution, Chromium-native, built-in waiting strategies
FastAPI over Flask/Django	Async-first, automatic OpenAPI docs, Pydantic integration, minimal boilerplate
Celery over Airflow	Lower overhead for this use case; Airflow is better suited for complex DAG-based ETL pipelines

Enhancement 6 — Updated System Architecture Diagram

The following diagram reflects the complete architecture including the Raw Data Storage Layer and Scraper Job Management System added in this enhancement pass.







What Changed From the Original Diagram

Addition	Location in Diagram
Raw Data Storage Layer	Between Data Collection and Validation
Scraper Job Management System	Embedded within Raw Data Storage Layer block
Technology labels	Added to each layer header
Data Versioning indicators	Shown as columns in the Storage layer block
Service API endpoints	Made explicit in the Service Layer block
End User personas	Named at the bottom of the diagram

Document Version: 1.1 — Original architecture preserved in full. Six engineering enhancements appended above.
Authored: 2026-03-06